

Source files:

dot_product.cpp

```
#include "dot_product.h"

void dot_product(din_t A[4], din_t B[4], dout_t *Y) {
#pragma HLS PIPELINE
#pragma HLS ARRAY_PARTITION variable=A complete
#pragma HLS ARRAY_PARTITION variable=B complete

    dout_t result = 0;

    // Compute dot product:  $Y = A_0*B_0 + A_1*B_1 + A_2*B_2 + A_3*B_3$ 
    for (int i = 0; i < 4; i++) {
#pragma HLS UNROLL
        result += (dout_t)(A[i] * B[i]);
    }

    *Y = result;
}
```

Dot_product.h

```
#ifndef DOT_PRODUCT_H
#define DOT_PRODUCT_H

#include "ap_int.h"

// Type definitions
typedef ap_int<8>  din_t;  // 8-bit signed input
typedef ap_int<16> dout_t; // 16-bit signed output

// Top function declaration
void dot_product(din_t A[4], din_t B[4], dout_t *Y);

#endif
```

Testbench file:

dot_product_tb.cpp

```
#include <iostream>
#include "dot_product.h"

int main() {
    din_t A[4], B[4];
    dout_t Y;
    int errors = 0;

    // Helper lambda to run one test
    auto run_test = [&](int a0, int a1, int a2, int a3,
                       int b0, int b1, int b2, int b3,
                       int expected, const char* label) {
        A[0]=a0; A[1]=a1; A[2]=a2; A[3]=a3;
        B[0]=b0; B[1]=b1; B[2]=b2; B[3]=b3;
        dot_product(A, B, &Y);
        bool pass = (Y == expected);
        std::cout << label << ": Y=" << Y
                   << " Expected=" << expected
                   << (pass ? " PASS" : " FAIL") << "\n";
        if (!pass) errors++;
    };

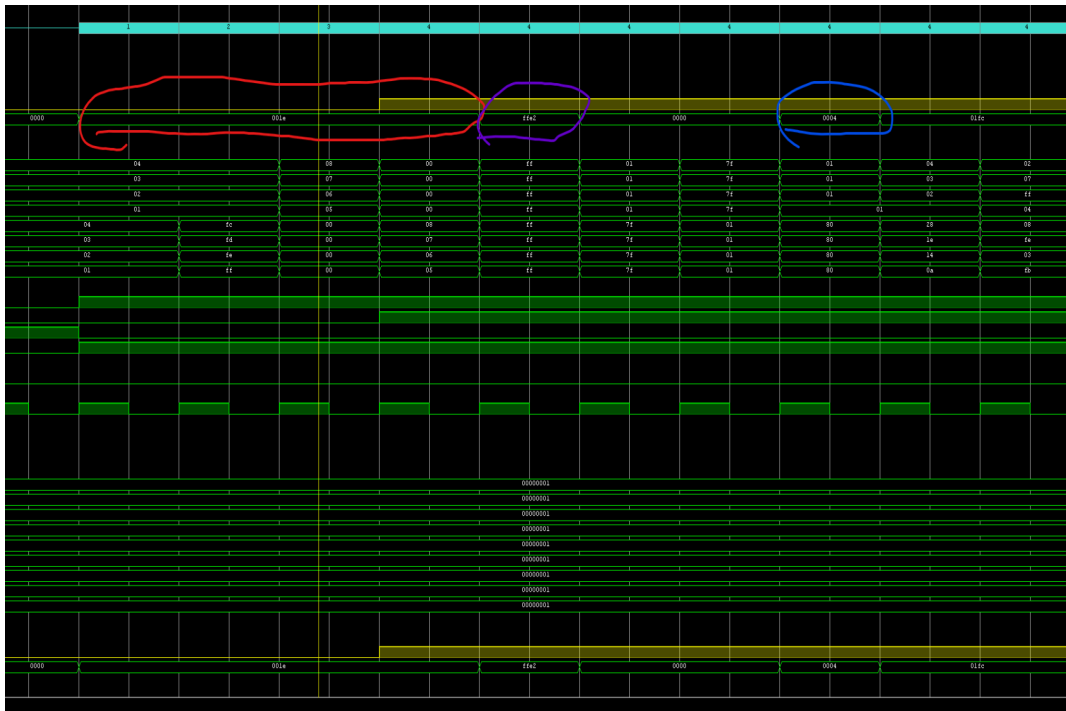
    // Test cases
    run_test( 1,  2,  3,  4,  1,  2,  3,  4,  30, "TC1 Basic
positive");
    run_test(-1, -2, -3, -4,  1,  2,  3,  4, -30, "TC2 Mixed signs");
    run_test( 0,  0,  0,  0,  5,  6,  7,  8,  0, "TC3 Zero vector
A");
    run_test( 5,  6,  7,  8,  0,  0,  0,  0,  0, "TC4 Zero vector
B");
    run_test(-1, -1, -1, -1, -1, -1, -1, -1,  4, "TC5 All negative");
    run_test(127,127,127,127, 1,  1,  1,  1, 508, "TC6 Max positive
A");
    run_test( 1,  1,  1,  1,127,127,127,127, 508, "TC7 Max positive
B");
    run_test(-128,-128,-128,-128, 1,  1,  1,  1, -512, "TC8 Min value");
    run_test( 10, 20, 30, 40, 1,  2,  3,  4, 300, "TC9 Larger
values");
    run_test(-5,  3, -2,  8, 4, -1,  7,  2, -21, "TC10 Mixed random");
```

```
std::cout << "\n" << (errors == 0 ? "ALL TESTS PASSED" : "SOME
TESTS FAILED")
    << " (" << errors << " errors)\n";

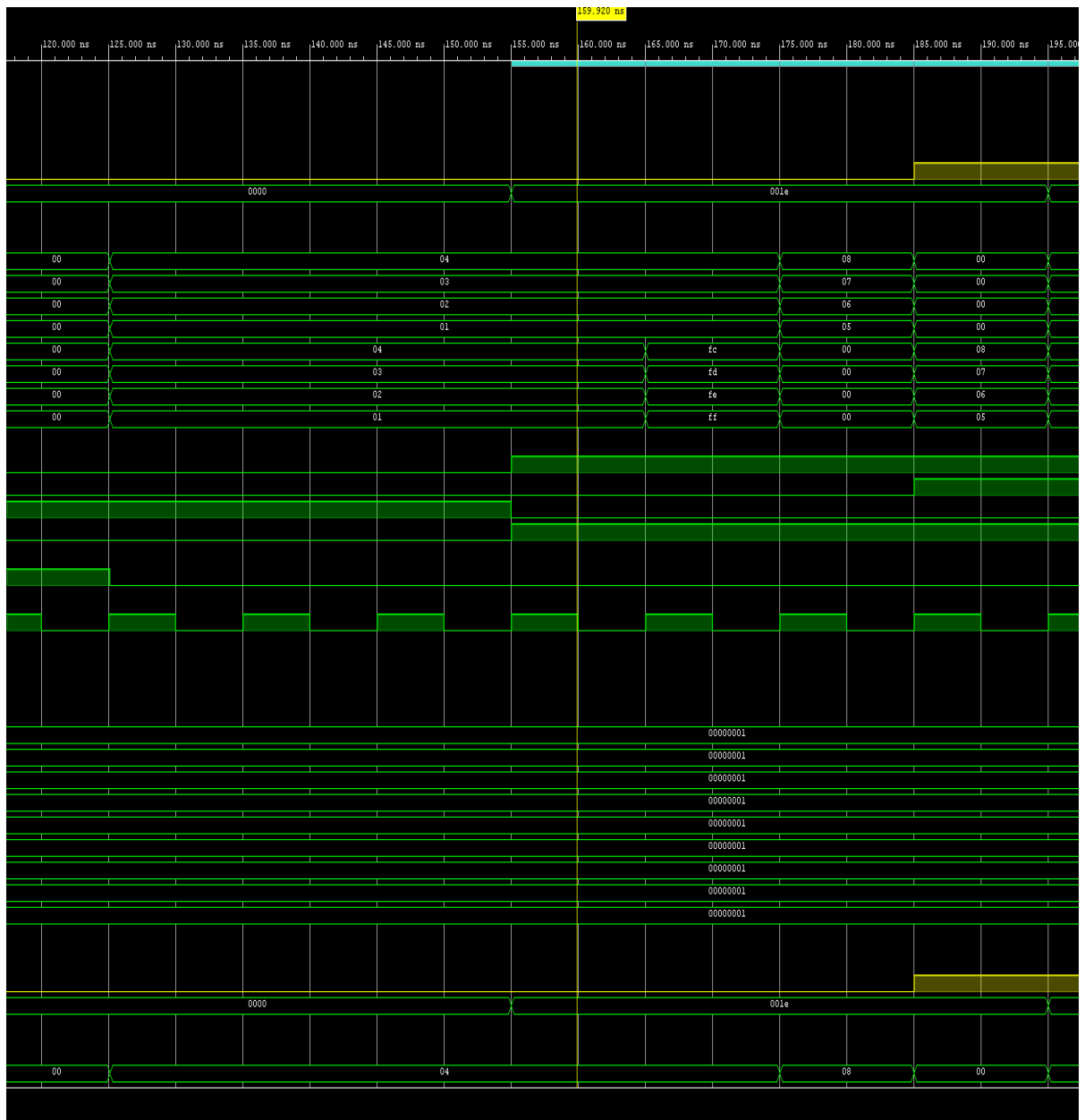
return errors;
}
```

Simulation waveforms for three cases:

General solutions to the three waveforms



1. Case 1: $1x_1 + 2x_2 + 3x_3 + 4x_4 = 30 = 001e$



Case 2: Two's complement
 $-1x1 - 2x2 - 3x3 - 4x4 = -30 = \text{ffe2}$

```
ff= -1
fe=-2
fd=-3
fc=-4
```


Project Report

Design choice: Why choose RTL vs. HLS?

HLS was chosen over RTL because it allows the dot product algorithm to be translated at a higher level in C++, which reduces development time while producing synthesizable RTL/ Since the core computation is a simple accumulation loop, HLS pragmas such as `#pragma HLS pipeline` and `#pragma HLS unroll` allow the compiler to automatically optimize the hardware without manual pipeline stage design. This is the ideal first implementation where correctness and faster iteration are important.

Architecture: Block diagram or flowchart of your implementation



Used draw.io for image above.

Verification: How did you verify correctness?

Correctness was verified through three stages:

- C Simulation: 10 test cases were run covering positive values, negative values, zero vectors, max/min 8-bit values, and mixed-sign inputs. All 10 test cases passed with expected outputs verified against manually computed results.
- C Synthesis: The design was synthesized, and the synthesis report was reviewed to confirm that latency, initiation interval, and resource usage were within expectations.
- C/RTL Co-Simulation: The generated RTL was re-verified against the C++ testbench. Waveforms were captured showing input vectors A and B, the clock signal, and the correct output Y across multiple test cases.

Reflection: What was the hardest part? What would you improve?

The hardest part of this project was honestly just getting the environment to cooperate. Setting up Vitis HLS took more time than expected. The clock uncertainty error in the config file and tracking down the right license for Co-Simulation were both frustrating roadblocks that had nothing to do with the actual design work. Once everything was configured correctly though, implementing the dot product itself was pretty straightforward since the algorithm is simple enough that the HLS pragmas handled most of the optimization automatically.

If I had more time, there are a few things I would go back and improve. The most interesting would be adding a streaming interface using AXI4-Stream ports so the module could actually be integrated into a larger SoC design rather than just sitting as a standalone component. I would also run two separate synthesis configurations, one focused on minimizing area and one focused on maximizing speed, to get a concrete comparison of how the pipeline pragma affects LUT and DSP usage. Finally, the testbench could use more thorough edge case coverage, particularly around alternating max and min values to stress test the overflow behavior of the 16-bit output.